AI-Based Resume Screening and Candidate Shortlisting System

Project Documentation | Ankita Kumari | B.Tech CSE (Data Science), ABES Engineering College (AKTU), Noida | Repository: https://github.com/Ankitakumari13/AI-BASED-RESUME-SCREENING

1. Overview

The system helps recruiters screen many resumes quickly. A recruiter pastes a job description (JD), uploads resumes (PDF, DOCX or TXT) and receives a ranked shortlist. Every candidate gets a match score, a skill-match percentage, the skills that matched or are missing, and the experience found. The shortlist can be downloaded as a CSV file.

The tool is decision support. It is trained on a synthetic dataset, so a human must always make the final hiring decision.

Technology: Python, Streamlit, scikit-learn (TF-IDF and Logistic Regression), pandas, NumPy, SciPy, joblib, pypdf, python-docx.

2. Key Features

- Upload multiple resumes in PDF, DOCX or TXT format.
- Automatic text extraction; empty, scanned or unreadable files are skipped with a warning.
- Anonymisation: name, email, phone numbers and links are removed before scoring.
- Match features: TF-IDF cosine similarity, skill overlap, years of experience and role match.
- Logistic Regression model gives a selection probability for each candidate.
- Ranked table, bar chart and a "Why this ranking?" explanation for every candidate.
- Settings for shortlist size (top N, 1 to 20) and an optional role (15 roles, or auto-detected from the JD).
- Built-in demo: one sample JD and six sample resumes.
- CSV download of the results.

3. Project Structure

File or folder	Purpose
generate_dataset.py	Generates the synthetic dataset (seed 42, 10,000 rows)
resume_screening_dataset.csv	The generated training dataset
resume_screening_ml.ipynb	Analysis, feature engineering, model comparison and evaluation
resume_screening_model.joblib	Trained model (copy at project root)
resume_screener_app/app.py	Streamlit front end
resume_screener_app/screening.py	Cleaning, anonymisation, feature engineering and scoring
resume_screener_app/resume_screening_model.joblib	Trained Logistic Regression and TF-IDF vectorizers used by the app
resume_screener_app/requirements.txt	Python dependencies
resume_screener_app/sample_jd.txt	Demo job description (Data Scientist)
resume_screener_app/sample_resumes/	Six demo resumes (candidate_1.txt to candidate_6.txt)

4. Getting Started

4.1 Prerequisites

- Python 3.11 or newer (scikit-learn 1.8.0 requires it).
- pip, and a modern web browser.

4.2 Installation

```
git clone https://github.com/Ankitakumari13/AI-BASED-RESUME-SCREENING.git
cd AI-BASED-RESUME-SCREENING/resume_screener_app
pip install -r requirements.txt
```

Dependencies: streamlit>=1.50, pandas, numpy, scipy, scikit-learn==1.8.0, joblib, pypdf, python-docx. scikit-learn is pinned to the version used for training so the saved model loads correctly.

4.3 Run the app

```
streamlit run app.py
```

Open the URL shown in the terminal, usually http://localhost:8501.

4.4 How to use

1. Paste a job description, or click **Load demo job + resumes** in the sidebar.
2. Upload resumes (PDF, DOCX or TXT).
3. Optionally set the shortlist size (top N) and a role in the sidebar.
4. Click **Screen candidates**.
5. Review the summary metrics, ranked table, bar chart and the per-candidate explanations.
6. Click **Download ranked shortlist (CSV)** to save shortlist.csv.

5. How It Works

1. **Input.** The app reads the JD text and the uploaded files. extract_text() uses pypdf for PDF, python-docx for DOCX and plain decoding for TXT.
2. **Anonymisation.** clean_resume() removes the candidate name, emails, phone numbers and links so the model cannot use them.
3. **Feature engineering.** For each resume and the JD the system computes TF-IDF cosine similarity, skill overlap against a vocabulary of 125 skills (with synonyms such as ml, sklearn, powerbi and nodejs), years of experience, text length and role match.
4. **Prediction.** A Logistic Regression model, trained on TF-IDF text plus the scaled match features, outputs the probability that the candidate should be selected.
5. **Ranking.** Candidates are sorted by probability. A score of 0.5 or higher is labelled Recommend. A candidate is Shortlisted when ranked within the top N and recommended.
6. **Output.** The app shows metrics, a table with progress bars, a bar chart, matched and missing skills per candidate, and the CSV download.

6. Code Reference

6.1 screening.py

Item	Description
ROLES	List of the 15 supported job roles
SKILLS	Vocabulary of 125 skills used for matching
SYN	Synonym map that normalises skill names (for example sklearn to scikit-learn)
normalise(text)	Lower-cases and normalises text and synonyms
skills_in(text)	Returns the skills from the vocabulary found in a text
years_exp(text)	Extracts years of experience from a text
clean_resume(text)	Anonymises a resume (name, email, phone, links)
infer_role(jd)	Detects the most likely role from a job description
extract_text(data, filename)	Extracts text from PDF, DOCX or TXT bytes
Screener	Loads the .joblib model; Screener.score(resumes, names, job_description, role) returns the ranked results table and the skills found in the JD

6.2 app.py

The Streamlit page has a settings sidebar (top N, role, demo buttons), two input areas (JD text and resume upload), a Screen candidates button, and a results section. The model is loaded once with st.cache_resource, and results are kept in st.session_state so the page does not recompute on every interaction.

6.3 Output columns

Column	Meaning
Rank	Position after sorting by match probability

Column	Meaning
Candidate	File name of the resume
Match score	Model probability from 0 to 1
Skill match	Share of JD skills found in the resume
Experience (yrs)	Years of experience found
Recommendation	Recommend if the score is 0.5 or higher
Matched skills / Missing skills	Explanation lists
Shortlisted	True if in the top N and recommended

7. Dataset

The dataset is synthetic, created by generate_dataset.py (seed 42). Each row is one candidate resume screened against one job description. The select or reject decision comes from a hidden weighted match score (required and preferred skills, experience, education, projects, certifications and role fit) plus random noise, so the label is learnable but not trivial.

Property	Value
File	resume_screening_dataset.csv
Rows	10,000
Columns	10
Job roles	15
Unique job descriptions	1,500
Label balance	4,500 select and 5,500 reject

Columns include Candidate_ID, Resume, Job_Description, Role, Years_Experience, Education_Level, Num_Projects, Num_Certifications, Reason_for_decision and Decision.

Reason_for_decision states the outcome and is not used for training, because it would leak the label (it alone gave 100% accuracy).

To regenerate the dataset: run python generate_dataset.py from the project root.

8. Model Training and Evaluation

Training and evaluation are in resume_screening_ml.ipynb. The data is split 80/20 with stratification (random_state 42), giving 2,000 test resumes. Six models were compared:

Model	Accuracy	Precision	Recall	F1	ROC-AUC
LogReg (text + match features)	0.856	0.833	0.850	0.842	0.936
HistGradientBoosting (match features)	0.859	0.839	0.850	0.844	0.936
Linear SVM (text + match)	0.856	0.835	0.847	0.841	0.935
Random Forest (match features)	0.857	0.842	0.840	0.841	0.934
LogReg (TF-IDF text only)	0.690	0.662	0.639	0.650	0.758
Complement Naive Bayes (text)	0.602	0.560	0.533	0.546	0.644

Key findings:

- Text alone is a weak signal; the engineered match features lift accuracy from about 69% to about 86%.
- Five-fold cross-validation ROC-AUC: Logistic Regression on text 0.756 (plus or minus 0.011); gradient boosting on match features 0.936 (plus or minus 0.005).
- Ranking quality: Precision@10 per role averages 0.973, against 0.450 for a random baseline.
- Logistic Regression is deployed because its accuracy is close to the best model and it is fast and easy to explain.

9. Demo Results

Running the built-in demo (sample Data Scientist JD, six sample resumes) gives:

Candidate	Match score	Skill match
candidate_5	99.9%	100%
candidate_4	99.7%	100%
candidate_6	74.6%	50%

Candidate	Match score	Skill match
candidate_3	74.4%	62.5%
candidate_1	3.2%	62.5%
candidate_2	0.03%	37.5%

Scoring these six resumes takes about 0.1 seconds.

10. Deployment

The app can run locally or on Streamlit Community Cloud. For Streamlit Cloud, connect the GitHub repository, choose the folder that contains app.py and requirements.txt (resume_screener_app) and set app.py as the main file. Keep the scikit-learn pin in requirements.txt so the model loads.

11. Limitations and Responsible Use

- The model is trained on synthetic data; validate it on real hiring data before real use.
- Only text-based resumes are supported. Scanned (image-only) PDFs are skipped because they have no text layer.
- Skill matching uses a fixed vocabulary of 125 skills, and resumes are assumed to be in English.
- Scores are decision support only; a human makes the final decision.
- Personal details are removed before scoring, and uploaded resumes are held in memory for the session. This supports the principles of the Digital Personal Data Protection Act, 2023, but it is not a legal compliance guarantee.

12. Troubleshooting

Problem	Likely cause and fix
Model fails to load or version warning	scikit-learn version differs from 1.8.0. Run pip install -r requirements.txt
No text found warning for a PDF	The PDF is scanned or image-only. Use a text-based PDF or DOCX

Problem	Likely cause and fix
Error asking for a job description or resume	Paste a JD and upload at least one resume, or load the demo
No known skills detected in the JD	The JD names no skill from the vocabulary; add explicit skill names
Command streamlit not found	Install dependencies in the active Python environment and restart the terminal

13. Future Work

- Validate and retrain on real, consented hiring data.
- Add OCR for scanned resumes.
- Run bias and fairness audits on the scores.
- Expand the skill vocabulary and support more languages.
- Integrate with Applicant Tracking Systems.

14. References

- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- Salton, G. and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513-523.
- Streamlit Documentation: https://docs.streamlit.io
- Government of India (2023). The Digital Personal Data Protection Act, 2023.